

Beginner-C Bootcamp 5주차 과제

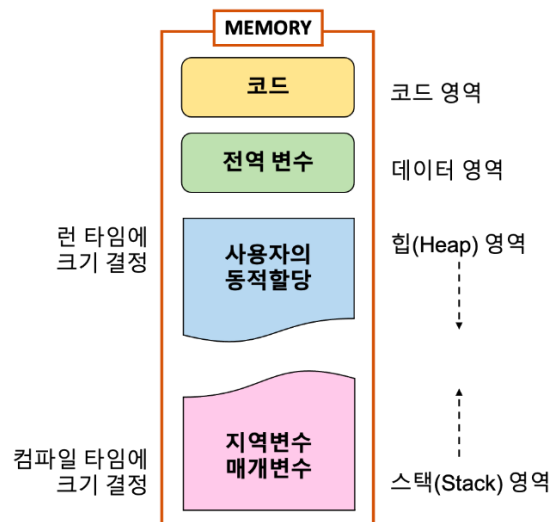
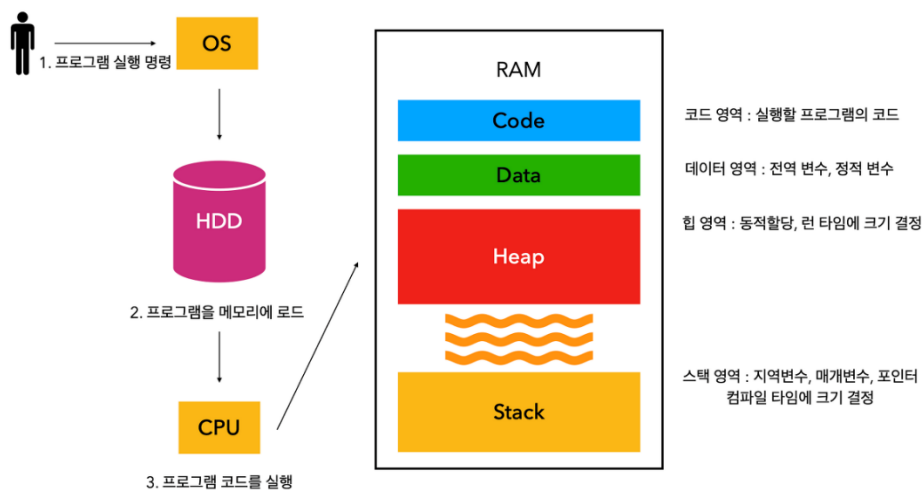
20215789 산업보안학과 김문정

Challenge 1. 메모리 레이아웃 이해하기

참고자료:

<https://velog.io/@shindoyeon/%EC%9A%B4%EC%98%81%EC%B2%B4%EC%A0%9C-%EB%A9%94%EB%AA%A8%EB%A6%AC-%EA%B5%AC%EC%A1%B0>

우선 메모리와 메모리 구조에 대해 간단하게 찾아보았다. 메모리란 CPU가 작업할 데이터를 빠르게 저장하고 사용하는 임시 공간을 말한다. 프로그램이 실행되기 위해선 먼저 프로그램이 메모리에 로드되어야 한다. 컴퓨터의 운영체제는 프로그램의 실행을 위해 다양한 메모리 공간을 제공하고, 각각의 메모리 공간은 상호 작용하며 프로그램 실행에 기여한다. 아래와 같이 프로그램이 실행되면 메모리에 로드되고, OS는 메모리(RAM)에 공간을 할당해준다.



메모리는(RAM) 텍스트(Text), 데이터(Data), 스택(Stack), 힙(Heap) 영역으로 나뉜다.

[Text]

코드를 실행하기 위해 저장되어있는 영역(= 코드 영역)

프로그램을 실행시키기 위해 구성되는 것들이 저장되는 영역

명령문들이 저장(제어문, 함수, 상수들이 저장)

[Data]

전역변수, 정적변수 등이 저장되는 공간

메인(main)함수 전(프로그램 실행 전)에 선언되어 프로그램이 끝날 때까지 메모리에 남아 있는 변수

Data영역도 크게 두 가지로 나뉨→초기화 된 변수 영역(initialized data segment), 초기화 되지 않은 변수 영역(uninitialized data segment)

초기화되지 않은 변수 영역: BSS(Block Started by Symbol)

[Heap]

사용자에 의해 관리되는 영역

동적으로 할당할 변수들이 여기에 저장

대개 '낮은 주소에서 높은 주소로 할당(적재)

[Stack]

함수를 호출할 때 지역변수, 매개변수들이 저장되는 공간

메인(main) 함수안에서의 변수들도 포함, 함수가 종료되면 해당 함수에 할당된 변수들을 메모리에서 해제(=Stack 자료구조의 pop)

Heap영역과 반대로 높은주소에서 낮은주소로 메모리에 할당

```
user@user-VirtualBox: ~  
#include <stdio.h>  
#include <stdlib.h>  
  
//Data_initialized, BSS  
  
int global_var=2; //initialized  
int uninit; //BSS  
  
int main(){  
    //Stack  
    int local_var=10;  
  
    //Heap(malloc)  
    int *heap_var=(int*)malloc(sizeof(int));  
    *heap_var=20;  
  
    //Text(Code)  
    char text[]="Hello";  
    //or char *text="Hello";  
  
    printf("=== Address===");  
    printf("global_init (Data_init): %p\n", &global_var);  
    printf("global_uninit (Data_BSS): %p\n", &uninit);  
    printf("local_var (Stack): %p\n", &local_var);  
    printf("heap_var (Heap): %p\n", &heap_var);  
    printf("text (Text): %p\n", (void *)text);  
}
```

각각의 변수 주소를 프린트해보면, 전역변수(데이터구역, 초기화됨): 0x555555558010

BSS구역(데이터구역_초기화 안됨): 0x555555558018

지역변수 구역(스택구역): 0x7fffffffd4

힙 구역: 0x7ffffffd8

텍스트 구역: 0x7ffffffde2

```
Apr 28 12:21
user@user-VirtualBox: ~
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
Files <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./memory...
(gdb) run
Starting program: /home/user/memory

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
=== Memory Address===
global_init (Data_init): 0x555555558010
global_uninit (Data_BSS): 0x555555558018
local_var (Stack): 0x7fffffffddd4
heap_var (Heap): 0x7fffffffddd8
text (Text): 0x7fffffffdde2
```

```
user@user-VirtualBox: ~
process 4718
Mapped address spaces:

   Start Addr           End Addr       Size     Offset Perms  objfile
   0x555555554000       0x555555555000     0x1000         0x0  r--p  /home/user
/memory
   0x555555555000       0x555555556000     0x1000     0x1000  r-xp  /home/user
/memory
   0x555555556000       0x555555557000     0x1000     0x2000  r--p  /home/user
/memory
   0x555555557000       0x555555558000     0x1000     0x2000  r--p  /home/user
/memory
   0x555555558000       0x555555559000     0x1000     0x3000  rw-p  /home/user
/memory
   0x555555559000       0x55555557a000     0x21000         0x0  rw-p  [heap]
   0x7ffff7c00000       0x7ffff7c28000     0x28000         0x0  r--p  /usr/lib/x
86_64-linux-gnu/libc.so.6
   0x7ffff7c28000       0x7ffff7db0000     0x188000     0x28000  r-xp  /usr/lib/x
86_64-linux-gnu/libc.so.6
   0x7ffff7db0000       0x7ffff7dff000     0x4f000     0x1b0000  r--p  /usr/lib/x
86_64-linux-gnu/libc.so.6
   0x7ffff7dff000       0x7ffff7e03000     0x4000      0x1fe000  r--p  /usr/lib/x
86_64-linux-gnu/libc.so.6
   0x7ffff7e03000       0x7ffff7e05000     0x2000      0x202000  rw-p  /usr/lib/x
--Type <RET> for more, q to quit, c to continue without paging--
```

gdb에서 info proc mappings 명령어를 쳐서 프로그램의 메모리 구역별 권한과 주소 범위를 확인한다. 우선 전역변수부터 확인해보면, 데이터 영역에 저장되어 있으며 0x555555558010에 저장되어있다. 이제 저 숫자가 포함되는 범위를 info proc mapping에서 확인해보면,

```
0x555555558000      0x555555559000      0x1000      0x3000  rw-p
/home/user/memory
```

이 변수가 왜 데이터 영역에 포함되는지 확인해보면, 권한 부분에서 rw-p가 있는데, 전역 변수는 값을 읽고 수정할 수 있어야 하기 때문에 rw권한이 부여된 것을 알 수 있다.

초기화되지 않은 변수인 BSS의 주소는 0x555555558018이며, 위 전역변수와 같은 Start Addr, End Addr 범위에 포함된다는 것을 알 수 있다. 이를 통해 BSS는 데이터구역 안에 포함된다는 것을 확인할 수 있었으며 초기화된 변수는 보통 데이터 영역의 앞부분에 저장되며, 초기화되지 않은 변수는 데이터 영역의 뒤쪽 어딘가에 저장된다는 것을 확인할 수 있었다.

gdb에서 info symbol 명령어를 통해 정확하게 확인해보면

```
--Type <RET> for more, q to quit, c to continue with
(gdb) info symbol 0x555555558010
global_var in section .data of /home/user/memory
(gdb) info symbol 0x555555558018
uninit in section .bss of /home/user/memory
```

다음과 같이 잘 저장된 것을 확인할 수 있다.

local_var(스택구역)의 주소는 0x7fffffffd4로 높은 주소 대역임을 확인할 수 있다.

```
0x7ffffffde000      0x7fffffffd000      0x21000      0x0  rw-p  [stack]
```

권한을 보면, 아까 데이터 영역과 마찬가지로 데이터를 읽고 수정(w)해야 하지만, 데이터가 코드로 실행되면 안 되므로 실행(x)은 금지된다는 것을 알 수 있다.

마찬가지로 info symbols로 어디 영역에 있는지 확인해보려 하였지만, 오류가 나서 AI의 도움을 받으니 info symbols명령어는 전역변수만 사용가능하다는 것을 알 수 있었다.

symbol 테이블에는 전역변수의 이름과 주소만 저장되어 있기 때문에, 지역변수로 symbols명령어는 확인이 불가하다는 사실을 알 수 있었다.

마지막 힙 구역과 텍스트 구역의 주소를 보면 0x7fffffffddd8, 0x7fffffffdde2로 스택 구역에 저장된 것을 볼 수 있었는데, 분명히 malloc과 문자열로 저장했음에도 왜 저런 결과가 나오는지 궁금하여 AI의 도움을 받았다.

내가 쓴 코드는 다음과 같은데,

```
int *heap_var=(int*)malloc(sizeof(int));
```

```
*heap_var=20;
```

```
char text[]="Hello";
```

```
//or char *text="Hello";
```

```
printf("heap_var (Heap):          %p\n", &heap_var);
```

```
printf("text (Text):              %p\n", (void *)text);
```

힙의 경우 마지막에 printf를 할 때 &heap_var를 한 것은, 힙 주소를 담고 있는 포인터 변수 그 자체가 어디 있는지를 묻는 것이고, 포인터 변수는 지역 변수이므로 스택 주소가 나오게 된다는 것을 알 수 있었다. 따라서 올바르게 적으려면, ampersand를 뺀 변수 이름만 적어야 할당받은 힙 주소가 출력된다는 것을 알 수 있었다.

텍스트 구역(문자열 리터럴)의 경우 char text[] = "Hello"; 이렇게 배열로 선언하였는데, 배열로 선언할 경우 문자열 Hello를 스택에 만든 배열 공간으로 복사해오기 때문에 주소를 프린트할 경우 스택 주소가 나오게 된다는 것을 알게 되었다. 따라서 올바르게 적으려면, char *text="Hello"; 또는 const char *text = "Hello"; 로 포인터로 선언하고, printf("%p, text);라고 적어야 텍스트 영역 주소가 출력된다는 것을 알 수 있었다.

코드를 위와 같이 다시 수정하여 다시 컴파일러를 돌려보면

```
This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
=== Memory Address===
global_init (Data_init): 0x555555558010
global_uninit (Data_BSS): 0x555555558018
local_var (Stack): 0x7fffffffddd4
heap_var (Heap): 0x5555555592a0
text (Text): 0x555555556008
```

주소는 다음과 같고, 다시 info proc mapping을 통해 확인해보자.

먼저 힙 영역의 주소는 0x5555555592a0이고,

```
0x555555559000      0x555555557a000      0x21000      0x0  rw-p  [heap]
```

권한을 보면, 동적으로 데이터를 저장하고(w) 읽어야(r) 하니까 rw이고, 코드가 아니니까 실행은 못 하게(-) 막아둔 개인 공간(p)인 것을 알 수 있다. 또한 주소를 보면 데이터영역 아래, 스택영역 위에 존재함을 알 수 있다.

다음으로 텍스트 영역의 주소를 확인해보면 0x555555556008이고,

```
0x555555556000      0x555555557000      0x1000      0x2000  r--p
/home/user/memmm
```

권한을 보면, r--p인데 바뀌지 않아야 하는 상수나 문자열을 저장하는 영역이므로 쓰기가 금지된다는 것을 알 수 있다. 또한 주소를 보면, 가장 낮은 주소에 매핑됨을 알 수 있다.

따라서 전체 메모리 구조는

[낮은 주소]

0x555555556008 → Code (Text)

↓

0x555555558010 → Data

0x555555558018 → BSS

↓

0x5555555592a0 → Heap (↑ 증가)

↓

(빈 공간)

↓

0x7fffffffddd4 → Stack (↓ 감소)

[높은 주소]

이렇게 구성된다는 것을 확인할 수 있었다.

Challenge 2. 버퍼 오버플로우 실험

gcc -g -O0 -fno-stack-protector 에서 뒤 fno-stack-protector가 무엇인지 궁금해 찾아보니 스택 보호 기능을 제거하는 옵션으로 버퍼 오버플로우 발생 시 프로그램이 강제 종료되지 않고 그냥 값이 덮이게 된다는 것을 알 수 있었다.

```
/usr/bin/ld: /tmp/ccDC4cYQ.o: in function `vuln':  
/home/user/vulnn.c:8:(.text+0x33): warning: the `gets' function is dangerous  
should not be used.  
user@user-VirtualBox:~$ ./vulnn  
AAA  
user@user-VirtualBox:~$ ./vulnn  
AAAAAAAAAAAAAAAAAAAA  
user@user-VirtualBox:~$ ./vulnn  
AAAAAAAAAAAAAAAAAAAA  
Segmentation fault (core dumped)  
user@user-VirtualBox:~$
```

길게 입력하면 다음과 같은 오류를 출력하며 프로그램이 종료된다.

```
Breakpoint 1, vuln () at vulnn.c:8  
8      gets(buf);  
(gdb) info frame  
Stack level 0, frame at 0x7fffffffddfd0:  
rip = 0x5555555518f in vuln (vulnn.c:8); saved rip = 0x555555551b5  
called by frame at 0x7fffffffde00  
source language c.  
Arglist at 0x7fffffffddde0, args:  
Locals at 0x7fffffffddde0, Previous frame's sp is 0x7fffffffddfd0  
Saved registers:  
rbp at 0x7fffffffddde0, rip at 0x7fffffffddde8  
(gdb)
```

buf의 위치를 확인하기 위해 info frame를 입력해보면 위 사진과 같이 나온다.

이를 해석해보면

frame at 0x7fffffffddfd0는 현재 프레임의 시작 주소(또는 기준점)/스택의 최상단

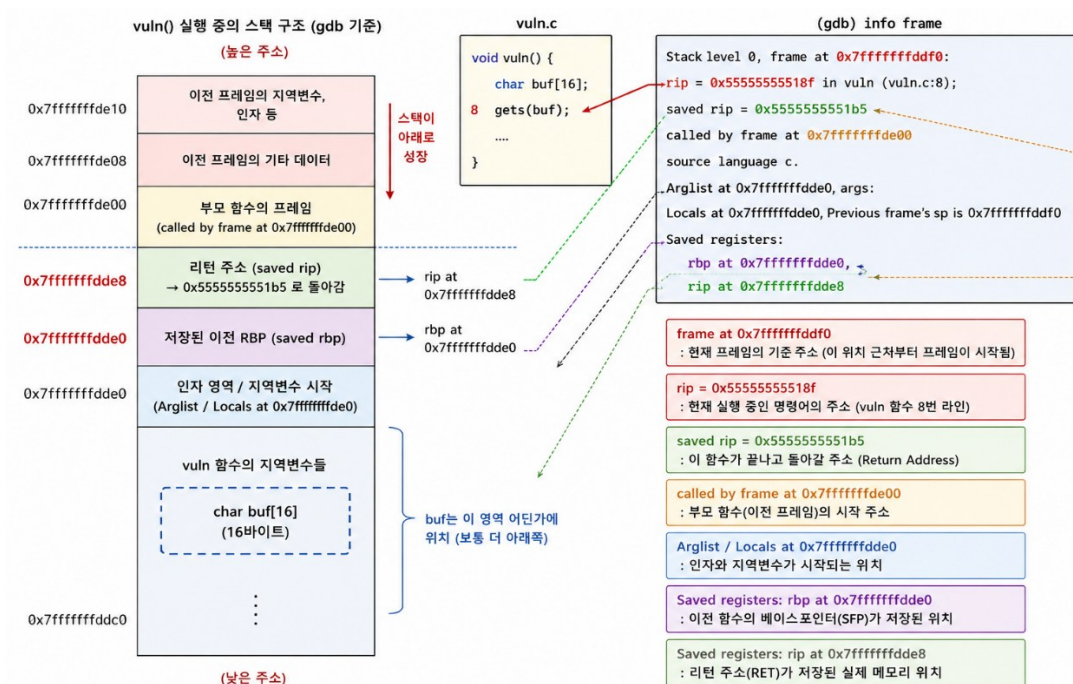
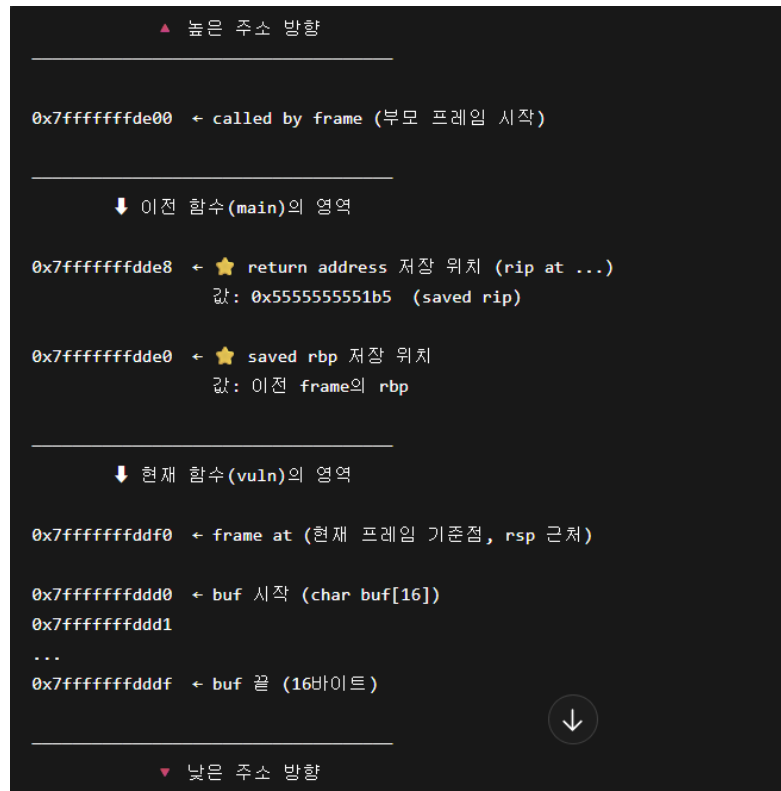
rip = 0x5555555518f in vuln은 vuln에서 현재 실행중인 위치(스택),

saved rip = 0x555555551b5는 리턴 주소(main으로 돌아갈 주소)를 의미한다는 것을 알 수 있다.

called by frame at 0x7fffffffde00는 부모 함수(이전 프레임)의 주소이며

rbp at 0x7fffffffde0은 이전 함수의 베이스 포인터(SFP, Saved Frame Pointer)가 저장된 위치 / 현재 스택 프레임의 기준점(바닥)을 의미한다.

rip at 0x7fffffffde8은 리턴 주소(RET)가 저장된 실제 메모리 위치라는 것을 알 수 있다.



위 그림을 보면, 우리가 덮고 싶은 것은 return address이고 이를 덮기 위해서는, 스택은 아래에서부터 위로 쌓이므로 현재 입력 시작 위치인 buf[16]+저장된 이전 rbp(즉, 호출한 main함수의 rbp)범위를 넘은 입력이 들어와야 한다는 것을 알 수 있다. 현재 buf는 16바이트이고, saved rbp와 saved rip의 주소 차이를 보면 8바이트이므로 24바이트를 초과한 입력에 대하여 ret add에 값이 덮이게 된다는 것을 알 수 있다.

```
(gdb) x/gx 0x7fffffffddde8
0x7fffffffddde8: 0x00005555555551b5 ) 이전
(gdb) c
Continuing.
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.
0x00005555555551a2 in vuln () at vulnn.c:9
9      }
(gdb) x/gx 0x7fffffffddde8
0x7fffffffddde8: 0x4141414141414141 ) 이후
(gdb)
```

덮어쓰기 이전, x/gx로 ret add가 저장된 rip값인 0x7fffffffddde8에 있는 값을 확인해보면 주소가 잘 나오는 것을 확인할 수 있다. 그러나 이후 24바이트가 넘는 긴 입력을 넣고, x/gx로 rip에 저장되어 있는 값을 확인해보면, 0x4141414141414141이라는 값이 저장됨을 알 수 있다. 16진수 41은 10진수 65며, 이를 아스키문자로 변환하면 A이므로 사용자가 입력한 값인 A가 ret add에 덮인 것을 확인할 수 있다.

또한, 이와 비슷하게 시크릿 함수를 불러오려면 어떻게 해야 하는지 궁금하여 AI의 도움을 받았더니, 바로 위와 마찬가지로 ret address에 시크릿 함수의 주소를 넣으면 시크릿 함수로 점프하게 된다는 것을 알 수 있었다. 또한, 시크릿 함수의 주소를 넣을 때 x86_64는 리틀엔디안 표기를 하므로 뒤에서부터 거꾸로 주소를 넣어야 한다는 것을 알 수 있었다.

이제 왜 gets() 함수가 버퍼 오버플로우에 취약한지 알아보자.

gets()의 가장 큰 문제는 입력받을 크기를 물어보지 않는다는 점이다. gets()는 사용자가 엔터를 칠 때까지 메모리에 데이터를 계속 넣기 때문에, 프로그래머가 char buf[16]이라고 16바이트 공간만 만들어두어도 사용자가 100바이트를 입력하면 gets()는 아무런 제재 없이 100바이트를 모두 쓴다. 따라서, 앞서 살펴본 것처럼 스택의 Saved RBP와 Saved RIP를 순식간에 덮어버리게 되는 결과가 발생한다. (이 때문에 최신 C 표준에서는 이 함수를 아예 삭제하거나 사용을 강력히 권고하지 않는다)

gets()이외에도 버퍼 오버플로우에 취약한 함수들은 다음과 같다.

1. scanf("%s", ..): %s 포맷 스트링은 공백 문자가 나올 때까지 입력을 받는데, 이때 입력 크기를 제한하지 않는다. 따라서 scanf("%9s", buf)처럼 숫자를 넣어 길이를 제한해야 한다.
2. strcpy(dest, src): src 문자열의 길이를 확인하지 않고 dest로 복사한다. 만약 src가 dest 보다 크면 바로 오버플로우가 발생한다. 따라서 복사할 길이를 지정하는 strncpy()를 사용해야 한다.
3. strcat(dest, src): 기존 문자열 뒤에 새 문자열을 이어 붙이는데, 합쳐진 결과가 dest의 전체 용량을 초과하는지 체크하지 않는다. 따라서, 2번과 마찬가지로 이어붙일 길이를 지정하는 strncat()를 사용해야 한다.
4. sprintf(buf, "format", ...): 서식에 맞춰 문자열을 만드는데, 결과물이 buf의 크기를 넘어가도 멈추지 않는다. 따라서, 버퍼의 크기를 인자로 받는 snprintf()를 사용해야 한다.